Learned Index Structures: Recent Advances and Challenges

Barna Berekméri
Eötvös Loránd University (ELTE)
Budapest, Hungary
barna.berekmeri@gmail.com

Abstract

Index structures are fundamental to database performance. Recently, the idea of replacing traditional data structures with machine learning models has gained attention. This paper surveys recent progress in learned indexes, focusing on learned Bloom filters, recursive models, and multidimensional extensions.

Keywords

Learned index, Bloom filter, LSM-tree, multidimensional indexing, databases

ACM Reference Format:
Barna Berekméri. 2025. Learned Index Structures: Recent Advances and Challenges. In . ACM, New York, NY, USA, 2 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Index structures are central to the performance of database management systems, enabling efficient data access and query execution. Traditional index structures, such as B-trees for range queries, hash tables for point lookups, and Bloom filters for approximate membership checks, have been refined over decades and serve as the base of most commercial systems. These data structures provide predictable worst-case guarantees and are well understood in terms of their asymptotic complexity. However, the exponential growth of data volumes, the increasing heterogeneity of workloads, and the demand for low-latency data-intensive applications have exposed limitations of classical index designs.

The fundamental challenge is that traditional indexes treat the key distribution as unknown or adversarial, which means that they often waste memory and compute resources to handle worst-case scenarios that rarely occur in practice. In contrast, real-world data distributions are typically skewed, correlated, or otherwise structured. This observation has motivated the exploration of machine learning (ML) as a tool to exploit such structure for indexing. The idea of *learned indexes* is that indexes can be replaced, or at least augmented, by predictive models that approximate the mapping between keys and their storage positions. If the model can be trained to predict positions or membership efficiently, then the cost of lookups and the memory overhead of index structures can be significantly reduced [4].

Conference'17, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

This idea represents a paradigm shift: rather than viewing an index as a rigid data structure, we view it as a learned function that leverages the statistical properties of the data. Over the past five years, this line of research has generated intense interest in both theory and system communities. Numerous papers in SIGMOD, VLDB, and ICDE have explored new learned index structures, extensions to multidimensional data, hybrid architectures that combine models with traditional indexes, and practical evaluations in storage engines [1–3, 5, 6].

Despite promise, learned indexes also pose serious challenges. They often lack worst-case guarantees, struggle with dynamic updates, and can be costly to train or adapt when the data distribution changes. Moreover, integrating learned indexes into storage engines such as LSM trees or into disk-based systems raises new questions about I/O complexity, cache behavior, and model management. In the following, we provide background and review recent contributions in this area.

2 Background and Related Work

2.1 The Learned Index Paradigm

The concept of learned indexes was introduced by Kraska et al. [4], who observed that a key-to-position mapping in a sorted array can be approximated by a cumulative distribution function (CDF). In this view, a B-tree traversal can be replaced by computing a model prediction and probing near the predicted location. This led to recursive model indexes (RMIs), where the upper levels of the hierarchy choose which model to use in the lower levels, mimicking the branching behavior of a tree, but in functional form.

The original proposal demonstrated significant performance improvements in lookup latency and memory footprint, sometimes by orders of magnitude. However, subsequent studies also revealed weaknesses: RMIs can suffer from high prediction errors in regions of the data distribution, leading to expensive verification steps, and their update performance is often inferior to that of B trees.

2.2 Multidimensional Learned Indexes

Extending learned indexes to multidimensional data is an active area of research. Workloads such as spatial queries, multiattribute range queries, or nearest-neighbor search demand indexes that operate over two or more dimensions. Classical solutions include kd-trees, R-trees, and their many variants. Learned approaches such as Flood, LISA, and RSMI attempt to partition the multidimensional space and use models to predict data placement.

The empirical study "How Good Are Multidimensional Learned Indexes?" provides the most comprehensive evaluation to date [2]. The authors benchmark a wide range of learned and traditional multidimensional indexes on synthetic and real-world data. They find that learned indexes can outperform traditional structures in

terms of memory efficiency and range query latency, sometimes by up to three orders of magnitude. However, the benefits are less clear for updates and high-dimensional queries such as neighbors k nearest, where traditional structures remain more robust. These findings highlight both the potential and the current limitations of the learned approaches.

2.3 Learned Bloom Filters

Another important line of research concerns Bloom filters, which are probabilistic data structures for approximate membership queries. Bloom filters are widely used in storage systems, including as part of LSM-trees, to avoid unnecessary disk accesses. Traditional Bloom filters guarantee no false negatives, but allow false positives with a probability determined by their size and hash function choice.

The concept of learned Bloom filters replaces or augments the hash functions with a classifier that predicts whether a key is likely to exist in the set. A Bloom backup filter ensures the correctness by capturing false negatives of the classifier [4]. Recent advances include partitioned learned Bloom filters (PLBFs), which allocate memory adaptively across partitions of the key space. However, the original construction of PLBFs was computationally expensive. Sato and Matsui propose faster construction algorithms with theoretical guarantees [5]. More recently, cascaded learned Bloom filters (CLBFs) use multiple classifier stages to achieve better balance between filter size and prediction accuracy, reducing memory usage by about 24% and accelerating negative lookups by up to 14× [6].

The integration of learned Bloom filters into storage engines is particularly noteworthy. Fidalgo and Ye propose two methods to embed the Bloom filters learned into LSM trees [3]. The first method uses classifiers to skip certain filter checks, thereby reducing latency, while the second replaces Bloom filters with learned models plus small backup filters, saving 70–80% of memory. These experiments demonstrate substantial performance improvements while preserving correctness guarantees.

2.4 Construction Complexity and Maintenance

While most early work on learned indexes focused on query performance, a critical practical issue is the cost of constructing and maintaining the models. Training a learned index on a large dataset can be prohibitively expensive if not carefully optimized. The recent paper "Can Learned Indexes be Built Efficiently?" investigates this problem in depth [1]. The authors analyze the complexity of different model construction strategies and show how sampling and incremental learning can mitigate training costs. They argue that unless construction complexity is carefully managed, the practical utility of learned indexes will be limited.

Another challenge is adapting to evolving data distributions. Static models may degrade in accuracy as new data arrives or distributions shift, leading to increased error rates and higher query costs. This underscores the need for incremental training methods or hybrid approaches that combine models with robust fallback mechanisms.

3 Conclusion

Learned indexes offer an exciting new perspective on a decades-old problem in database systems. By leveraging machine learning to approximate index functionality, they achieve significant gains in memory and query performance. Recent work has extended the paradigm to multidimensional queries and Bloom filters, and has begun to address the practical challenges of construction complexity and integration into storage engines. At the same time, open challenges remain in ensuring robustness under updates, managing distribution drift, and providing worst-case guarantees. These open problems define a rich agenda for future research at the intersection of databases and machine learning.

References

[1] ... 2024. Can Learned Indexes be Built Efficiently? A Deep Dive into Construction Complexity. *Proceedings of SIGMOD* (2024). doi:10.1145/3654919
[2] ... 2024. How Good Are Multi-dimensional Learned Indexes? An Experimental Study. *VLDB Journal* (2024). doi:10.1007/s00778-024-00893-6
[3] R. Fidalgo and J. Ye. 2025. Learned LSM-trees: Two Approaches Using Learned Bloom Filters. *arXiv preprint arXiv:2508.00882* (2025).
[4] Tim Kraska, Alex Beutel, Ed H. Chi, Jeff Dean, and Neoklis Polyzotis. 2018. The Case for Learned Index Structures. In *Proceedings of the 2018 ACM SIGMOD International Conference on Management of Data*. 489–504. doi:10.1145/3183713.3196909
[5] S. Sato and Y. Matsui. 2024. Fast Construction of Partitioned Learned Bloom Filter with Theoretical Guarantees. *arXiv preprint arXiv:2410.13278* (2024).
[6] S. Sato and Y. Matsui. 2025. Cascaded Learned Bloom Filter for Optimal Model-Filter Size Balance and Fast Rejection. *arXiv preprint arXiv:2502.03696* (2025).